# Unit II – Functions and Classes & Objects: Answers

## Section A – Debugging Questions

### Question 1: Dangling Reference in Return-by-Reference

#### 1. What happens to result (Line 8) – Undefined Behaviour (Runtime)

result is a local (automatic) variable that lives in the activation record (stack frame) of getMaximumScaled. When the function returns, the frame is popped and that stack memory is released for reuse. The function returned a reference to that dead location, so danglingRef is a **dangling reference**. Reading it (Line 19/27) is undefined behaviour: the next calls (cout, getArrayElement) overwrite that stack slot, so the value is garbage or the program crashes. Compilers warn: reference to local variable 'result' returned.

#### 2. Why getArrayElement is safe

arr[index] refers to an element of the array owned by the **caller** (data in main), which outlives the call. Returning a reference to an object whose lifetime extends beyond the function is safe and allows lvalue use (getArrayElement(data,1) = 999). result dies with the function, so returning a reference to it is unsafe. Rule: only return references to non-local objects (globals, statics, members, caller-owned data).

#### 3. Corrected code (return by value)

```
#include <iostream>
int getMaximumScaled(int a, int b, int scaleFactor) {
   int maxVal = (a > b) ? a : b;
   return maxVal * scaleFactor;          // return by value
}
int& getArrayElement(int arr[], int index) {
   return arr[index];                    // refers to caller's array: safe
}
```

```
int main() {
    int x = 10, y = 20;
    int scaled = getMaximumScaled(x, y, 2);
    std::cout << "Scaled Max: " << scaled << std::endl;
    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999;
    std::cout << "Updated Array[1]: " << data[1] << std::endl;
    std::cout << "Stored Scaled Value: " << scaled << std::endl;
    return 0;
}
```

**4. Dry run / output**

| Step | Action | Value |
|---|---|---|
| 1 | maxVal = max(10,20) | 20 |
| 2 | 20 × 2 returned by value | 40 copied into scaled |
| 3 | getArrayElement(data,1)=999 | data = {100, 999, 300} |
| 4 | scaled printed again | 40 (stable) |

**Output:**

Scaled Max: 40

Updated Array[1]: 999

Stored Scaled Value: 40

**Question 2: Static Data Member Linkage & Static Member Function Scope**

**1. Linker error**

static double totalBankReserves; inside the class is only a **declaration**: it says the member exists but allocates no storage (there is one shared copy for the whole class, not one per object). Exactly one definition outside the class is required, in one translation unit: double

BankAccount::totalBankReserves = 0.0;. Without it the constructor's += references a symbol that never gets defined, so the linker reports undefined reference to 'BankAccount::totalBankReserves'.

**2. Compilation error on Line 19**

A static member function belongs to the class, not to any object, so it has **no this pointer**. accHolder is a per-object (non-static) member, and displayAudit() cannot know which object's accHolder is meant, so the compiler reports invalid use of non-static data member. A static function can only use static members (or objects passed to it).

**3. Corrected code**

```
#include <iostream>
#include <string>
class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves;
public:
    BankAccount(std::string name, double initialDeposit) {
        accHolder = name;
        balance = initialDeposit;
        totalBankReserves += initialDeposit;
    }
    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $" << totalBankReserves << std::endl;
    }
    void displayHolder() const {
        std::cout << "Account Holder: " << accHolder << ", Balance: $" << balance << std::endl;
    }
```

```
};
double BankAccount::totalBankReserves = 0.0;   // definition: allocates storage
int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);
    BankAccount::displayAudit();
    acc1.displayHolder();
    acc2.displayHolder();
    return 0;
}
```

**Output:**

```
Total Reserves in Vault: $8000
Account Holder: Alice, Balance: $5000
Account Holder: Bob, Balance: $3000
```

**Question 3: Overload Ambiguity via Default Arguments**

**1. Line 19 – computeVolume(5.0)**

Overload 1 (double, double=1.0, double=1.0) can be called with one argument through its defaults, and Overload 2 (double) also takes exactly one argument. Both match with an **exact** match on the argument, so neither is better and the call is **ambiguous**. Default arguments make a function viable for several arities; they do not rank lower during overload resolution.

**2. Line 22 – computeVolume(3.0, 7.0)**

Yes, it is also ambiguous. Overload 3 (double radius, double height) matches exactly, but Overload 1 matches too with width = 7.0 and height = 1.0 (default). Both have identical exact conversions for the two supplied arguments, so there is no unique best viable function. Matching process: (1) collect candidates by name; (2) keep viable ones (right argument count, counting defaults); (3) rank by conversion quality per argument; (4) if no single best exists → ambiguity error.

**3. Refactor**

Make each overload differ in **number of parameters** with no overlapping defaults: 1 argument = cube, 2 = cylinder, 3 = cuboid. (Alternatively give them different names, e.g. cuboidVolume with defaults.)

**4. Corrected code**

```
#include <iostream>
double cubeVolume(double side) { return side * side * side; }
double cylinderVolume(double radius, double height) { return 3.14159 * radius * radius * height
; }
double cuboidVolume(double length, double width = 1.0, double height = 1.0) { return length *
width * height; }
// Alternative keeping the name computeVolume (distinct arity, no defaults):
double computeVolume(double side) { return cubeVolume(side); }
double computeVolume(double radius, double height) { return cylinderVolume(radius, height); }
double computeVolume(double l, double w, double h) { return cuboidVolume(l, w, h); }
int main() {
    std::cout << "Volume: " << computeVolume(5.0) << std::endl;
    std::cout << "Cylinder: " << computeVolume(3.0, 7.0) << std::endl;
    std::cout << "Cuboid: " << computeVolume(2.0, 3.0, 4.0) << std::endl;
    return 0;
}
```

Calculation: cylinder = $3.14159 \times 3^2 \times 7 = 197.92017$, printed as 197.92 with default 6 significant digits.

**Output:**

```
Volume: 125
Cylinder: 197.92
Cuboid: 24
```

**Question 4: Array of Objects Missing Default Constructor**

**1. The rule**

The compiler synthesizes an implicit default constructor **only if the class declares no constructors at all**. Once any constructor (here Student(int, std::string)) is defined, the implicit one is not generated. Student batch[3]; must default-construct 3 elements, so it fails: no matching function for call to 'Student::Student()'.

**2. Two techniques**

- **Technique A:** add an explicit default constructor Student() : rollNumber(0), name("Unknown") {} (or give all parameters default values); then assign real data later.
- **Technique B:** use an initializer list so every element is built with the parameterized constructor: Student batch[3] = { Student(101,"Aarav"), Student(102,"Bhavna"), Student(103,"Chirag") }; No default constructor is needed.

**3. Corrected program (Technique A)**

```
#include <iostream>
#include <string>
class Student {
private:
   int rollNumber;
   std::string name;
public:
   Student() : rollNumber(0), name("Unknown") {}          // Technique A
   Student(int r, std::string n) : rollNumber(r), name(n) {}
   void display() const {
      std::cout << "Roll: " << rollNumber << ", Name: " << name << std::endl;
   }
};
int main() {
```

```
    Student batch[3];
    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");
    for (int i = 0; i < 3; ++i) batch[i].display();
    return 0;
}
```

**Output:**

Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag

**Question 5: Friend Function Declaration & Scope Flaws**

**1. alertThreshold on Line 29**

inspectDiagnostics is a **non-member** function. A friend function is not a member, so it has **no this pointer** and no implicit object. Names of class members cannot be used unqualified; each access must name an object: a.alertThreshold. Friendship only grants permission to read the private members, not implicit access.

**2. Line 41**

A friend function is not a member of Engine, so v8.inspectDiagnostics(...) is invalid ('class Engine' has no member named 'inspectDiagnostics'). It is called like an ordinary function: inspectDiagnostics(v8, audit);.

**3. Forward declaration**

Inside Engine the friend declaration mentions the type TelemetryAuditor, which the compiler has not seen yet. The original code forward-declared Engine (a typo for the wrong class). class TelemetryAuditor; must appear before class Engine. A forward declaration is enough because

the friend declaration only needs the name (parameters here are references, so the full type is not required until the function body).

**4. Corrected code**

```
#include <iostream>
class TelemetryAuditor;                    // forward declaration
class Engine {
private:
   int temperature;
   int rpm;
public:
   Engine(int t, int r) : temperature(t), rpm(r) {}
   friend void inspectDiagnostics(const Engine &e, const TelemetryAuditor &a);
};
class TelemetryAuditor {
private:
   int alertThreshold;
public:
   TelemetryAuditor(int threshold) : alertThreshold(threshold) {}
   friend void inspectDiagnostics(const Engine &e, const TelemetryAuditor &a);
};
void inspectDiagnostics(const Engine &e, const TelemetryAuditor &a) {
   if (e.temperature > a.alertThreshold) {
      std::cout << "Warning: Engine overheat!" << std::endl;
   } else {
      std::cout << "Engine operating nominally at " << e.rpm << " RPM." << std::endl;
   }
}
int main() {
   Engine v8(105, 3200);
   TelemetryAuditor audit(100);
```

```
    inspectDiagnostics(v8, audit);
    return 0;
}
```

Dry run: temperature 105 > threshold 100 → overheat branch.

**Output:**

Warning: Engine overheat!

**Section B – Pseudo-Code Questions**

**Question 1: BranchOffice with Static Data & Static Functions**

**Design overview**

- Static attributes totalVaultReserves and totalBranchesActive are shared by all branches and initialized once outside the class.
- Instance methods update both the branch balance and the shared static total, keeping them consistent.
- Static methods use only static data (no THIS) and are called via the class name.

```
CLASS BranchOffice
  PRIVATE:
    STRING  branchID, branchName
    REAL    currentBalance
    STATIC REAL    totalVaultReserves
    STATIC INTEGER totalBranchesActive

  PUBLIC:
    CONSTRUCTOR BranchOffice(STRING id, STRING name, REAL openingDeposit)
      SET branchID = id
      SET branchName = name
```

```
  SET currentBalance = openingDeposit
  SET totalVaultReserves = totalVaultReserves + openingDeposit
  SET totalBranchesActive = totalBranchesActive + 1
ENDCONSTRUCTOR

METHOD Deposit(REAL amount)
  IF amount > 0 THEN
    SET currentBalance = currentBalance + amount
    SET totalVaultReserves = totalVaultReserves + amount
  ENDIF
ENDMETHOD

METHOD Withdraw(REAL amount) RETURNS BOOLEAN
  IF amount > 0 AND amount <= currentBalance THEN
    SET currentBalance = currentBalance - amount
    SET totalVaultReserves = totalVaultReserves - amount
    RETURN TRUE
  ELSE
    RETURN FALSE
  ENDIF
ENDMETHOD

STATIC METHOD GetVaultSummary()
  DISPLAY "Active branches: ", totalBranchesActive
  DISPLAY "Total reserves : ", totalVaultReserves
ENDMETHOD

STATIC METHOD GetGlobalLiquidityRatio(REAL targetReserve) RETURNS REAL
  IF targetReserve = 0 THEN
    RETURN 0
  ENDIF
```

```
    RETURN totalVaultReserves / targetReserve
  ENDMETHOD
ENDCLASS

SET BranchOffice.totalVaultReserves = 0.0       // static definition
SET BranchOffice.totalBranchesActive = 0

BEGIN Main
  BranchOffice b1("B001", "Downtown", 200000)
  BranchOffice b2("B002", "Uptown", 300000)
  b1.Deposit(150000)
  BOOLEAN ok
  SET ok = b2.Withdraw(50000)
  BranchOffice.GetVaultSummary()
  DISPLAY BranchOffice.GetGlobalLiquidityRatio(1000000)
END
```

**Assumption:** opening deposits of $200,000 (Branch1) and $300,000 (Branch2) since none were specified.

**Trace table**

| Step | Operation | Branch balance | totalVaultReserves | totalBranchesActive |
|---|---|---|---|---|
| 0 | Initial (static init) | – | 0 | 0 |
| 1 | Create Branch1 (200,000) | B1 = 200,000 | 200,000 | 1 |
| 2 | Create Branch2 (300,000) | B2 = 300,000 | 500,000 | 2 |

| Step | Operation | Branch balance | totalVaultReserves | totalBranchesActive |
|---|---|---|---|---|
| 3 | B1.Deposit(150,000) | B1 = 350,000 | 650,000 | 2 |
| 4 | B2.Withdraw(50,000): 50,000 ≤ 300,000 → TRUE | B2 = 250,000 | 600,000 | 2 |
| 5 | GetVaultSummary() | – | prints 600,000 | prints 2 |
| 6 | Liquidity ratio (target 1,000,000) | – | 600,000 / 1,000,000 = 0.6 | – |

**Question 2: Matrix2D with Friend Dot-Product**

**Design overview**

- 3×3 grid is a private array; access is only through bounds-checked methods.
- Members are declared in the class and defined outside with Matrix2D::.
- The friend function reads A.grid[i][i] and B.grid[i][i] directly.

**CLASS Matrix2D**

```
  PRIVATE:
    ARRAY grid[3][3] OF REAL
  PUBLIC:
    METHOD Initialize()
    METHOD SetElement(INTEGER row, INTEGER col, REAL value)
    METHOD GetElement(INTEGER row, INTEGER col) RETURNS REAL
    METHOD Display()
    FRIEND FUNCTION ComputeCrossTrace(Matrix2D A, Matrix2D B) RETURNS REAL
ENDCLASS
```

```
METHOD Matrix2D::Initialize()
  INTEGER i, j
  FOR i = 0 TO 2 STEP 1 DO
    FOR j = 0 TO 2 STEP 1 DO
      SET grid[i][j] = 0.0
    ENDFOR
  ENDFOR
ENDMETHOD

METHOD Matrix2D::SetElement(INTEGER row, INTEGER col, REAL value)
  IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
    SET grid[row][col] = value
  ELSE
    DISPLAY "Error: index out of range"
  ENDIF
ENDMETHOD

METHOD Matrix2D::GetElement(INTEGER row, INTEGER col) RETURNS REAL
  IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
    RETURN grid[row][col]
  ENDIF
  DISPLAY "Error: index out of range"
  RETURN 0.0
ENDMETHOD

METHOD Matrix2D::Display()
  INTEGER i, j
  FOR i = 0 TO 2 STEP 1 DO
    FOR j = 0 TO 2 STEP 1 DO
      DISPLAY grid[i][j], " "
```

```
    ENDFOR
    DISPLAY NEWLINE
  ENDFOR
ENDMETHOD

FUNCTION ComputeCrossTrace(Matrix2D A, Matrix2D B) RETURNS REAL   // friend
  REAL sum
  INTEGER i
  SET sum = 0.0
  FOR i = 0 TO 2 STEP 1 DO
    SET sum = sum + (A.grid[i][i] * B.grid[i][i])
  ENDFOR
  RETURN sum
ENDFUNCTION

BEGIN Main
  Matrix2D A, B
  A.Initialize()  B.Initialize()
  A.SetElement(0,0,2)  A.SetElement(1,1,4)  A.SetElement(2,2,6)
  B.SetElement(0,0,1)  B.SetElement(1,1,3)  B.SetElement(2,2,5)
  DISPLAY ComputeCrossTrace(A, B)
END
```

Trace (A diag = [2,4,6], B diag = [1,3,5])

| i | A.grid[i][i] | B.grid[i][i] | Product | sum |
|---|---|---|---|---|
| init | – | – | – | 0 |
| 0 | 2 | 1 | 2 | 2 |
| 1 | 4 | 3 | 12 | 14 |

| i | A.grid[i][i] | B.grid[i][i] | Product | sum |
|---|---|---|---|---|
| 2 | 6 | 5 | 30 | 44 |

Result: ComputeCrossTrace = **44.0**.

**Question 3: StudentRecord – Array of Objects & Bubble Sort**

**Design overview**

- Each object calculates its own percentage and grade (private CalculateResult).
- Bubble sort compares GetPercentage() and swaps whole objects; descending order, so swap when left < right.

**CLASS StudentRecord**

```
PRIVATE:
  INTEGER rollNo
  STRING  studentName
  ARRAY marks[3] OF REAL
  REAL    percentage
  CHARACTER grade
  METHOD CalculateResult()
    SET percentage = (marks[0] + marks[1] + marks[2]) / 3
    IF percentage >= 85 THEN
      SET grade = 'A'
    ELSE
      IF percentage >= 70 THEN
        SET grade = 'B'
      ELSE
        IF percentage >= 50 THEN
          SET grade = 'C'
        ELSE
          SET grade = 'F'
```

```
        ENDIF
      ENDIF
    ENDIF
  ENDMETHOD
 PUBLIC:
  CONSTRUCTOR StudentRecord()
    SET rollNo = 0
    SET studentName = ""
    SET marks[0] = 0, marks[1] = 0, marks[2] = 0
    SET percentage = 0
    SET grade = ' '
  ENDCONSTRUCTOR
  METHOD ReadData(INTEGER r, STRING name, REAL m1, REAL m2, REAL m3)
    SET rollNo = r
    SET studentName = name
    SET marks[0] = m1
    SET marks[1] = m2
    SET marks[2] = m3
    CalculateResult()
  ENDMETHOD
  METHOD GetPercentage() RETURNS REAL
    RETURN percentage
  ENDMETHOD
  METHOD Display()
    DISPLAY rollNo, studentName, percentage, grade
  ENDMETHOD
ENDCLASS

BEGIN Main
 ARRAY S[3] OF StudentRecord
 INTEGER i, pass
```

```
  StudentRecord temp
  S[0].ReadData(1, "Asha",  60, 70, 80)
  S[1].ReadData(2, "Ravi",  90, 85, 95)
  S[2].ReadData(3, "Meena", 40, 50, 45)
  FOR pass = 0 TO 1 STEP 1 DO                  // n-1 passes
    FOR i = 0 TO 1 - pass STEP 1 DO
      IF S[i].GetPercentage() < S[i+1].GetPercentage() THEN
        SET temp = S[i]
        SET S[i] = S[i+1]
        SET S[i+1] = temp
      ENDIF
    ENDFOR
  ENDFOR
  FOR i = 0 TO 2 STEP 1 DO
    S[i].Display()
  ENDFOR
END
```

**Sample data (assumed):** Asha 60/70/80 → 70.0 (B); Ravi 90/85/95 → 90.0 (A); Meena 40/50/45 → 45.0 (F).

Dry-run trace

| Pass | i | Compare | Swap? | Array after step |
|---|---|---|---|---|
| 1 | 0 | Asha 70 < Ravi 90 | Yes | Ravi, Asha, Meena |
| 1 | 1 | Asha 70 < Meena 45 ? No | No | Ravi, Asha, Meena |
| 2 | 0 | Ravi 90 < Asha 70 ? No | No | Ravi, Asha, Meena |

Final rank list: 1. Ravi 90.0 (A), 2. Asha 70.0 (B), 3. Meena 45.0 (F).

**Question 4: Flexible Logger – Overloading, Default Args, Inline**

**Design overview**

- FormatPrefix is INLINE (tiny, called on every log).
- Overload 1: one STRING. Overload 2: message plus defaulted severity and timestamp. Overload 3: (INTEGER, STRING) selected by argument types.
- **Ambiguity caveat:** with the specified defaults, a one-STRING call matches Overload 1 and Overload 2. The pseudo-code adopts the rule 'a signature needing no defaults beats one that needs defaults'. In real C++ this call would be ambiguous (see Unit II Debugging Q3), so there you would remove the default of severityLevel.

```
INLINE FUNCTION FormatPrefix(STRING prefixTag) RETURNS STRING
  RETURN "[" + prefixTag + "] "
ENDFUNCTION

FUNCTION LogEvent(STRING message)                          // Overload 1
  DISPLAY FormatPrefix("INFO") + message
ENDFUNCTION

FUNCTION LogEvent(STRING message, INTEGER severityLevel = 1,       // Overload 2
                  STRING timestamp = "00:00:00")                   // chosen when 2 or 3 arguments are gi
ven
  STRING severityText
  IF severityLevel = 1 THEN
    SET severityText = "LOW"
  ELSE
    IF severityLevel = 2 THEN
      SET severityText = "MEDIUM"
    ELSE
      IF severityLevel = 3 THEN
```

```
      SET severityText = "CRITICAL"
    ELSE
      SET severityText = "UNKNOWN"
    ENDIF
  ENDIF
 ENDIF
 DISPLAY "[" + severityText + "] @ " + timestamp + " : " + message
ENDFUNCTION

FUNCTION LogEvent(INTEGER errorCode, STRING systemModule)        // Overload 3
 DISPLAY "[HARDWARE_FAULT] Module: " + systemModule +
       " -> Error Code: 0x" + ToHex(errorCode)
ENDFUNCTION

BEGIN Main
 LogEvent("System booted")
 LogEvent("Disk usage high", 2)
 LogEvent("Overheat detected", 3, "14:30:05")
 LogEvent(255, "ADC_MODULE")
END
```

**Call resolution trace**

| Call | Overload chosen | severityLevel | timestamp | Output |
|---|---|---|---|---|
| LogEvent("System booted") | 1 (exact, 1 arg) | – | – | [INFO] System booted |
| LogEvent("Disk usage high", 2) | 2 (2 args) | 2 (given) | "00:00:00" (default) | [MEDIUM] @ 00:00:00 : Disk usage high |

| Call | Overload chosen | severityLevel | timestamp | Output |
|---|---|---|---|---|
| LogEvent("Overheat detected", 3, "14:30:05") | 2 (3 args) | 3 (given) | "14:30:05" (given) | [CRITICAL] @ 14:30:05 : Overheat detected |
| LogEvent(255, "ADC_MODULE") | 3 (INTEGER, STRING) | – | – | [HARDWARE_FAULT] Module: ADC_MODULE -> Error Code: 0xFF |

**Question 5: RegisterBank – Return by Reference**

**Design overview**

- GetRegister returns a reference (alias) to the real storage, so a call can sit on the left of <-.
- An invalid index returns a reference to the dummy errorSink, so a bad write never touches other memory.

**CLASS RegisterBank**

```
 PRIVATE:
  ARRAY registers[8] OF INTEGER
  INTEGER errorSink
 PUBLIC:
  CONSTRUCTOR RegisterBank()
    INTEGER i
    FOR i = 0 TO 7 STEP 1 DO
      SET registers[i] = 0
    ENDFOR
    SET errorSink = -1
  ENDCONSTRUCTOR

  METHOD GetRegister(INTEGER index) RETURNS REF INTEGER
```

```
    IF index >= 0 AND index < 8 THEN
      RETURN REF registers[index]
    ELSE
      DISPLAY "ALERT: register index out of range: ", index
      RETURN REF errorSink
    ENDIF
  ENDMETHOD

  METHOD DisplayState()
    INTEGER i
    FOR i = 0 TO 7 STEP 1 DO
      DISPLAY "R", i, " = 0x", ToHex(registers[i])
    ENDFOR
  ENDMETHOD
ENDCLASS

BEGIN Main
  RegisterBank bank
  INTEGER readVal
  bank.GetRegister(0) <- 170
  bank.GetRegister(1) <- 255
  SET readVal = bank.GetRegister(0)
  DISPLAY "Read R0 = ", readVal
  bank.GetRegister(10) <- 999          // goes to errorSink
  bank.DisplayState()
END
```

**Trace**

| Statement | Action | Result |
|---|---|---|
| Construct | registers = 0 ×8; errorSink = -1 | all 0x00 |
| GetRegister(0) <- 170 | reference to registers[0]; write 170 | R0 = 0xAA |
| GetRegister(1) <- 255 | reference to registers[1]; write 255 | R1 = 0xFF |
| readVal = GetRegister(0) | read through reference | readVal = 170 |
| GetRegister(10) <- 999 | 10 ≥ 8 → alert, reference to errorSink | errorSink = 999; registers unchanged |
| DisplayState() | | R0=0xAA, R1=0xFF, R2..R7=0x00 |

Mechanism of <code>func() = val</code>

A function that returns T& yields an **lvalue**: the call expression designates an existing object (it is an alias of the returned object, not a copy). The assignment operator needs an lvalue on its left, so GetRegister(0) = 170 stores 170 directly into registers[0]. A function returning by value yields a temporary rvalue, which cannot be assigned to. The returned object must outlive the call (here it is a member of the bank object), otherwise the reference would dangle.